

Alice Through the Word Embedding

**Parsimonious reflection of language
syntax and semantics**

Gary Nan Tie, July2, 2026

Abstract

For the purposes of next-token and sequence-to sequence learning, Alice is a parsimonious state machine that combines a recurrent implicit deep learning model transition with an attention-based output over learned fixed points, jointly optimized for syntactic prediction and semantic coherence within a word embedding space.

Synopsis

Preliminaries

A **word embedding** is typically a representation of a word in a vector space. The representation encodes the meaning of words in such a way, that words closer in the vector space are expected to be similar in meaning. For example, the word embedding $\text{word2vec}[1]$, captures information about the meaning of a word based on the surrounding words, syntactic co-occurrence.

Cosine distance, the metric given by:

$\text{Cos}(a, b) := \langle a, b \rangle / (\|a\| \|b\|)$, is a measure of similarity.

Let $\text{Emb}: \mathbb{R}^p \rightarrow \mathbb{R}^{p'}$ be a chosen word embedding, mapping token t_i to vector v_i , $i \in [m]$.

A **state machine** (S, O) begins in an initial state h_0 , that gets updated with each successive input t_i , via $h_i = S(h_{i-1}, t_i)$ where S is a transition function, and then outputs $o_{i+1} = O(h_i)$, where O is an output function. (The right index shift is for autoregressive continuation.) Diagrammatically, picture a state machine as a spine with vertebrae hidden states h and ribs either side t - h - o , token to state to output.

Let $\theta = \{ (W_1, b_1), (W_2, b_2) \}$ be parameters

for a **feedforward network** defined by

$\text{FFN}_\theta(h_i) := \text{ReLU}(h_i W_1 + b_1) W_2 + b_2$

implicitRNN [2] functions similarly to a vanilla RNN, processing sequence inputs one by one. For each time step i , where $t_i \in \mathbb{R}^p$ represents the i -th token in a sequence $(t_1, t_2, \dots, t_m)$, the input u_i for implicitRNN is a concatenation of t_i and the previous hidden state h_{i-1} , akin to a vanilla RNN. The implicit prediction rule for implicitRNN is as follows:

$$h_0 = 0, k_0 = 0$$

$$u_i = [t_i \ h_{i-1}]'$$

$$k_i = \phi(Ak_i + Bu_i) \text{ (equilibrium equation)}$$

$$h_i = S(h_{i-1}, t_i) := Ck_i + Du_i \text{ (prediction equation)}$$

where $A := \Gamma - \text{diag}(|\Gamma| \mathbf{1}) + \gamma \text{Id}$, $0 < \gamma < 1$, ensures A well-posed for ϕ , that is, the fixed point equation has a solution and it is unique.

In this setup, the recurrent layer is replaced with an implicit structure comprising the equilibrium and prediction equations, where we learn [3] matrices A, B, C, D and fixed points k_i .

Alice

Is a state machine with transition and output functions (S, O) , where S is defined by the implicitRNN above, and state h_i is viewed as a contextual representation of token t_i . Given a token sequence $t_1, t_2, \dots, t_i$ the goal is to find a token continuation o_{i+1} with certain properties. The state machine output enables this by choosing the continuation o_{i+1} to be $O(h_i)$, defined by attention layers with the desired (perplexity and Renyi α -entropy) properties.

Attention layer

Input h_i .

Let score be a desired similarity function, like a reproducing kernel.

For $i \in [m]$, define probabilities

$$\alpha_{ij} := \text{soft max over } j \in [i] \text{ score}(h_i, k_j).$$

Let $h_i^\wedge = \sum_{j \in [i]} \alpha_{ij} v_j$ and define update $h_i' := \text{FFN}_\Theta(h_i^\wedge)$.

Output h_i' .

Stack L attention layers $\Theta = (\theta_1, \dots, \theta_L)$ composing updates and now let h_i' denote the final L -th update.

For $i \in [m]$, define probabilities

$$\beta_{ij} := \text{soft max over } j \in [i] \text{ score}(h_i', k_j)$$

and let $j^*i := \arg \max_{j \in [i], \beta_{ij}}$.

Define token continuation $o_{i+1} = \mathbf{O}(h_i) := t^{j^*i}$,
the token whose fixed point k to date is most similar
(probable under distribution β) to update h' .

Define the average Renyi α -entropy of $\{t_1, \dots, t_m\}$,
a measure of semantic alignment by:

$E(\{t_i\}) := (1/m) \sum_i H(i, \alpha)$,
where $H(i, \alpha) := H_\alpha(\beta_{ij})$, the Renyi α -entropy of $\{\beta_{ij}\}$.

Together perplexity of output $\mathbf{O}$ and average Renyi α -entropy E
train the h_i to holistically capture syntax and semantics.

Alice training problem

Let $\lambda > 0$, learn implicitRNN matrices A, B, C, D and fixed points k_i ,
as well as, feedforward network parameters Θ to minimize:

$$[\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\})]$$

Remarks:

$\mathbf{O}$ next-token continuation $t_1, \dots, t_i, o_{i+1}$ reflects the
word embedding through the $h_i^\wedge$.

Perplexity intrinsically evaluates the continuation,
regularized by average Renyi α -entropy.

Solving the Alice training problem, is solving the
next-token learning problem jointly constrained
by syntactic co-occurrence (perplexity) and
semantic alignment (Renyi α -entropy).

Next-token prediction

Initialization: Let t_1 = the empty token or a seed token

Given t_1 , define $t_2 = o_2$.

Given t_1, t_2 , define $t_3 = o_3$.

Given t_1, t_2, t_3 , define $t_4 = o_4$ and so on ...

autoregressively generating next-token continuation.

For **sequence-to-sequence learning**, we solve the next-token problem for both the target and source tokens thereby learning contextual token representations for target and source. We can then directly compare their state similarity to determine a best translation based on meaning rather than surface syntax alone.

Thus Alice is a parsimonious dynamic model of language syntax and semantics.

Ecila

is a sister state machine (S, O') ,
where $o'_i = O'(h_i) := \text{FFN}\theta'(h_i)$ approximates v_i .

In detail, the **Ecila training problem**

Learn implicitRNN matrices A, B, C, D and fixed points k_i for h_i ,
and feedforward network parameters θ' for o'_i to minimize:

$$\sum_{i \in \{m\}} \text{Cos}(o'_i, v_i)$$

So **Ecila** simulates Emb , the word embedding.

Aliceecila

Is the state machine (S, O, O') with two output functions,
mother to embedding Ecila and transformer sister Alice.

The **Aliceecila training problem**

Given $\lambda > 0$, learn matrices A, B, C, D and fixed points k_i ,
parameters Θ and θ' , to minimize:

$$\sum_{i \in \{m\}} \text{Cos}(o'_i, v_i) + [\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\})]$$

So we have three state machines Alice, Ecila and Aliceecila
from which to solve the next-token learning problem.

Estimation strategies for learning continuation O

1/ Use mini-batch SGD to learn Alice (S, O) or Alicecila (S, O, O') .

2/ Use a single layer Alice to pre-train a spine.
Then on the fly, in parallel, learn Alice ribs on top of this fixed spine.

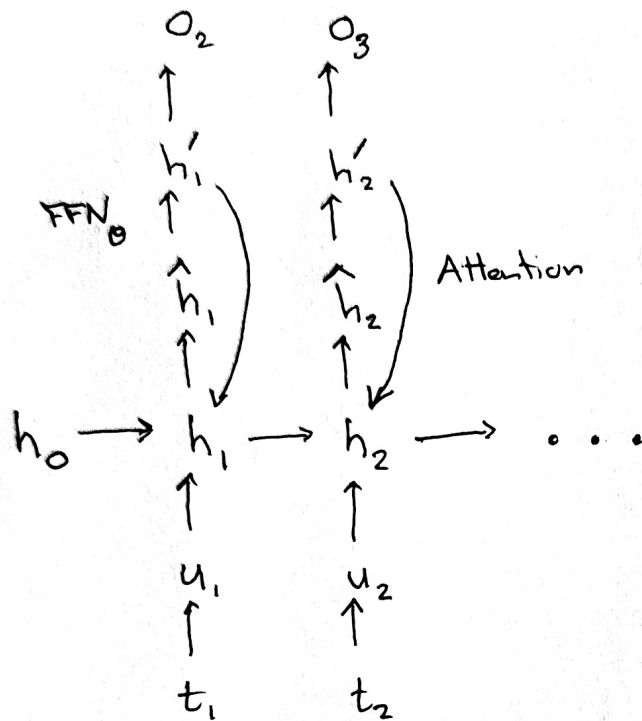
3/ Learn in stages a hybrid state machine.
First use Ecila to learn a spine (discard the ribs),
then on top of this pre-trained spine, use Alice layers
to learn ribs in parallel.

Each of these learning strategies represent a different
tradeoff between time and training problem faithfulness.

For the purposes of next-token and sequence-to sequence learning,
Alice is a parsimonious state machine that combines a recurrent
implicit deep learning model transition with an attention-based output
over learned fixed points, jointly optimized for syntactic prediction and
semantic coherence within a word embedding space.

"Curiouser and curiouser!"

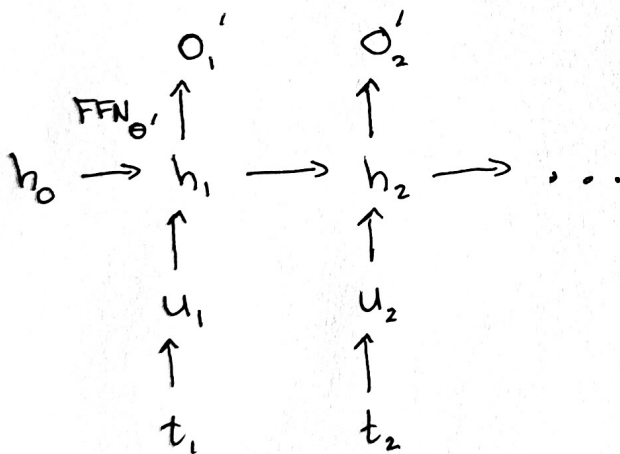
Appendix



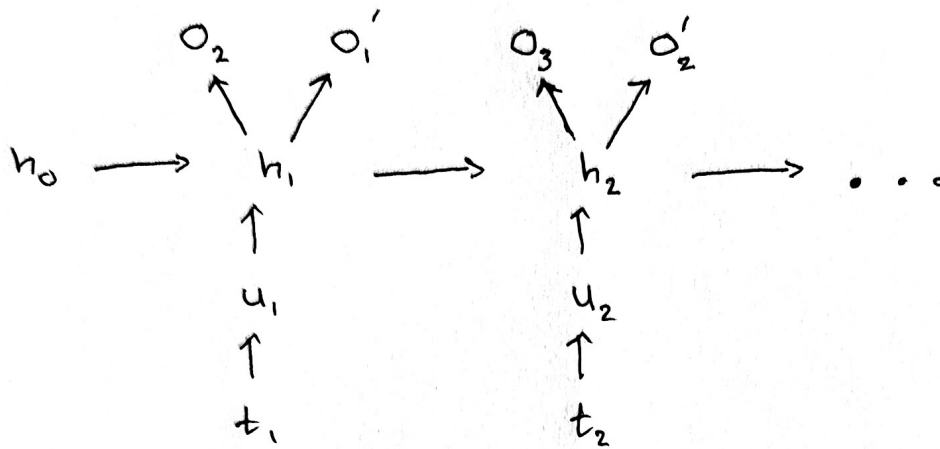
Alice ($\mathbb{S}, \mathbb{O}$)

- pre-train $\{h_i\}$

- learn $\{O_j\}$ in parallel



Ecila ($\mathbb{S}, \mathbb{O}'$)



AliceCila $(\mathcal{S}, \Theta, \Theta')$

Let $\Theta = \{(W_1, b_1), (W_2, b_2)\}$ be parameters
for a feedforward network defined by:

$$\text{FFN}_{\Theta}(h_i) \triangleq \text{ReLU}(h_i W_1 + b_1) W_2 + b_2$$

State Machine $(\mathcal{S}, \mathcal{O})$

Input: target tokens $t_1, \dots, t_m \in \mathbb{R}^p$

Initial state: $h_0 = 0 \in \mathbb{R}^q$

Next state: $h_i = \mathcal{S}(h_{i-1}, t_i) \in \mathbb{R}^q, \quad i \in [m]$

Output: $o_{i+1} = \mathcal{O}(h_i) \in \mathbb{R}^p, \quad i \in [m]$

where $\mathcal{S}$ is a transition function and

$\mathcal{O}$ is an output function.

—

Let $\text{Emb}: \mathbb{R}^p \rightarrow \mathbb{R}^{p'}$

$$t_i \mapsto v_i \quad i \in [m]$$

be a word embedding, like word2vec.

—

Cosine metric: $\text{Cos}(a, b) \triangleq \frac{\langle a, b \rangle}{\|a\| \|b\|}$

—

implicit RNN

$$h_0 = 0 \in \mathbb{R}^q \quad k_0 = 0 \in \mathbb{R}^q$$

$$u_i = \begin{pmatrix} t_i \\ h_{i-1} \end{pmatrix} \in \mathbb{R}^r, \quad r = p + q$$

$$k_i = \varphi(Ak_i + Bu_i) \in \mathbb{R}^q$$

$$h_i = \mathcal{S}(h_{i-1}, t_i) \triangleq Ck_i + Du_i \in \mathbb{R}^q$$

where $A \triangleq \mathcal{J} - \text{diag}(|\mathcal{J}| \mathbb{1}_q) + \gamma \text{Id}_q$, $0 < \gamma < 1$,

ensures A is well-posed for φ , that is,

the fixed point equation has a solution and

it is unique.

So an implicit RNN constructs $\mathcal{S}$.

Attention layer:

Input h_i .

Let score be a desired similarity function,
like a reproducing kernel.

For $i \in [m]$, define probabilities

$$\alpha_{ij} = \text{softmax}_{j \in [i]} \text{score}(h_i, k_j)$$

Let $\hat{h}_i = \sum_{j=1}^i \alpha_{ij} v_j$ and define update

$$h'_i = \text{FFN}_{\theta}(\hat{h}_i) \quad (\dim h'_i = \dim k_j)$$

Output h'_i

—//—

Stack L attention layers $\Theta \triangleq \{\theta_1, \dots, \theta_L\}$

composing updates FFN_{θ_l} and now let

h'_i denote the final L -th update.

After L layers, for $i \in [m]$ define probabilities

$$\beta_{ij} = \underset{j \in [J]}{\text{softmax score}}(h'_i, k_j) \text{ and}$$

$$\text{let } j_i^\beta = \underset{j \in [J]}{\arg \max} \beta_{ij}.$$

$$\text{Define token continuation } o_{i+1} = \Phi(h_i) \triangleq t_{j_i^\beta}$$

So attention layers construct Φ .

Define the average Renyi α -entropy of $\{t_1, \dots, t_m\}$ by:

$$E(\{t_i\}_{i=1}^m) \triangleq \frac{1}{m} \sum_{i=1}^m H_\alpha^i$$

$$\text{where } H_\alpha^i \triangleq H_\alpha(\{\beta_{ij}\}_{j=1}^J), \quad H_\alpha \text{ Renyi } \alpha\text{-entropy.}$$

Token continuation $t_1, t_2, \dots, t_i, o_{i+1}$ via:

Alice (S, Θ) training problem (next-token)

Let $\lambda > 0$, learn implicit RNN matrices A, B, C, D and fixed points k_i , as well as, feedforward network parameters Θ to minimize:

$$[\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\}_{i=1}^m)]$$

—

Next-token prediction

Initialization: let $t_1 = \text{empty token or a seed token}$

Given t_1 , define $t_2 = o_2$.

Given t_1, t_2 define $t_3 = o_3$

Given t_1, t_2, t_3 define $t_4 = o_4$ and so on, ...

—

NB: token continuation o_{i+1} is shaped

by perplexity and Renyi α -entropy.

Sequence-to-Sequence learning

target tokens $t_1, \dots, t_m$
Input: source tokens $\tilde{t}_1, \dots, \tilde{t}_m$

For $\{t_i\}$ and $\{\tilde{t}_j\}$ solve their respective

Alice next-token learning problem to find their

contextual representations $\{h'_i\}$ and $\{\tilde{h}'_j\}$

Define $S_{ij} = \underset{j \in [n]}{\text{softmax score}}(h'_i, \tilde{h}'_j)$, $i \in [m]$

and let $j_i^\delta = \underset{j \in [n]}{\arg \max} S_{ij}$.

Given target t_i , $i \in [m]$,

define source output $o_i \triangleq \tilde{t}_{j_i^\delta}$

So $t_1, \dots, t_m$ is translated to $o_1, \dots, o_m$

where $o_i \in \{\tilde{t}_j\}_{j=1}^n$.

—

Ecila ($\mathcal{S}, \Theta'$) training problem

Learn implicit RNN matrices A, B, C, D and

fixed points k_i for h_i , as well as, Feedforward

network parameters Θ' for O'_i to minimize:

$$\sum_{i=1}^m \text{Cos}(O'_i, v_i)$$

So Ecila simulates Emb, the word embedding.

→

AliceEcila ($\mathcal{S}, \Theta, \Theta'$) training problem

Given $\lambda > 0$, learn matrices A, B, C, D and

fixed points k_i , parameters Θ and Θ' to minimize:

$$\sum_{i=1}^m \text{Cos}(O'_i, v_i) + [\text{Perplexity}(o_2, \dots, o_{m+1}) + \lambda E(\{t_i\}_{i=1}^m)]$$

→

References

[1] 'Efficient Estimation of Word Representations in Vector Space'

Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean

<https://arxiv.org/abs/1301.3781>

[2] 'The Extrapolation Power of Implicit Models'

Juliette Decugis, Alicia Y. Tsai,

Max Emerling, Ashwin Ganesh, Laurent El Ghaoui

arXiv:2407.14430v1 [cs.LG] 19 Jul 2024

[3] 'Robust Implicit Networks via Non-Euclidean Contractions'

Saber Jafarpour, Alexander Davydov,

Anton V. Proskurnikov, Francesco Bullo

arXiv:2106.03194v6 [cs.LG] 26 Jan 2022